



BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH

ĐỒ ÁN MÔN HỌC

Lập Trình Mạng Máy Tính

ỨNG DỤNG PHÁT TRỰC TUYẾN NHẠC VỚI ĐỀ XUẤT TỰ ĐỘNG DỰA TRÊN SỞ THÍCH NGƯỜI DÙNG

Ngành: Công Nghệ Thông Tin

Lớp: 22DTHB4

Giảng viên hướng dẫn: Th.S Nguyễn Lê Văn

Sinh viên thực hiện:

Đỗ Gia Ân

Mã SV: 2280600147

Bùi Nhật Huy

Mã SV: 2280618692

Lê Minh Quý

Mã SV: 2280602656

Nguyễn Thị Phương Thùy

Mã SV: 2280603166

TP.Hồ Chí Minh, tháng 10 năm 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH

ĐỒ ÁN MÔN HỌC

Lập Trình Mạng Máy Tính

**ỨNG DỤNG PHÁT TRỰC TUYẾN NHẠC VỚI ĐỀ XUẤT TỰ
ĐỘNG DỰA TRÊN SỞ THÍCH NGƯỜI DÙNG**

Ngành: Công Nghệ Thông Tin

Lớp: 22DTHB4

Giảng viên hướng dẫn: Th.S Nguyễn Lê Văn

Sinh viên thực hiện:

Đỗ Gia Ân

Mã SV: 2280600147

Bùi Nhật Huy

Mã SV: 2280618692

Lê Minh Quý

Mã SV: 2280602656

Nguyễn Thị Phương Thùy

Mã SV: 2280603166

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TPHCM, Ngày.....tháng.....năm 2025

Giảng viên
(Ký tên, đóng dấu)

LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan rằng mọi thông tin và nghiên cứu được trình bày trong đồ án “Ứng dụng phát trực tuyến nhạc với đề xuất tự động dựa trên sở thích người dùng” là trung thực, khách quan, được thu thập và phân tích cẩn thận dựa trên các nguồn tài liệu chính thống và đáng tin cậy.

Bất kỳ thông tin hoặc ý kiến nào trích dẫn từ các nguồn khác đều được ghi rõ nguồn gốc và tuân thủ đúng quy định về trích dẫn. Nhóm cam kết không sao chép hoặc sử dụng thông tin không đúng đắn từ bất kỳ nguồn nào.

Đồ án này là công trình nghiên cứu chung của nhóm, được thực hiện bằng Java thuần và chưa từng được công bố tại bất kỳ nơi nào khác. Nhóm cam đoan đã tuân thủ đầy đủ các quy định về nghiên cứu, bao gồm việc tham khảo tài liệu, sử dụng dữ liệu và các công cụ phát triển phần mềm.

Nhóm hy vọng rằng đồ án “Ứng dụng phát trực tuyến nhạc với đề xuất tự động dựa trên sở thích người dùng” sẽ trình bày đầy đủ và chi tiết về hệ thống phát nhạc trực tuyến tích hợp tính năng gợi ý thông minh, đồng thời đóng góp một phần nhỏ vào việc nâng cao trải nghiệm nghe nhạc và ứng dụng công nghệ trong lĩnh vực giải trí trực tuyến.

TP.HCM, năm 2025

Nhóm thực hiện:

Đỗ Gia Ân

Bùi Nhật Huy

Lê Minh Quý

Nguyễn Thị Phương Thùy

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN

LỜI CAM ĐOAN

MỤC LỤC	1
CÁC TỪ VIẾT TẮT	3
DANH MỤC HÌNH ẢNH	5
DANH MỤC BẢNG	6
CHƯƠNG 1: TỔNG QUÁT	7
1.1 Giới thiệu	7
1.2 Nhiệm vụ.....	7
1.2.1 Tính cấp thiết và lý do chọn đề tài.....	7
1.2.2 Mục tiêu nghiên cứu	8
1.2.3 Đối tượng và phạm vi nghiên cứu	9
1.3 Cấu trúc.....	9
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	11
2.1 Nền tảng & công nghệ	11
2.1.1 Spring Boot 3, Java 17 và Maven	11
2.1.2 SQL Server.....	11
2.1.3 JPA/Hibernate.....	12
2.1.4 Thymeleaf	15
2.1.5 HTTP Live Streaming (HLS).....	16
2.1.6 JSON Web Token và bảo mật	16
2.1.7 Phương pháp đề xuất tự động	17
2.2 Kiến trúc.....	17
2.3 Thiết kế hệ thống	18
2.3.1 Mô hình Use case Diagram	18
2.3.2 Class Diagram.....	21
CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM	22
3.1 Triển khai hệ thống.....	22
3.1.1 Cấu hình build/run (Maven, application.properties)	22
3.1.2 Tạo CSDL SQL Server, mapping entity	22

3.1.3 Chạy ffmpeg và luồng xử lý HLS	24
3.2. Tính năng đã hoàn thành	25
3.3. Đánh giá hệ thống	28
CHƯƠNG 4 KẾT LUẬN VÀ KIẾN NGHỊ	29
4.1. Kết luận	29
4.2. Kiến nghị	29
TÀI LIỆU THAM KHẢO	31

CÁC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
HLS	HTTP Live Streaming: Giao thức phát trực tuyến thích ứng cho âm nhạc, giúp đảm bảo chất lượng phát ổn định và mượt mà.
MP3	MPEG-1 Audio Layer III: Định dạng tệp âm thanh được sử dụng trong hệ thống.
LTS	Long-Term Support: Phiên bản hỗ trợ dài hạn, dùng để chỉ phiên bản Java 17.
JVM	Java Virtual Machine: Máy ảo Java, môi trường chạy ứng dụng.
API	Application Programming Interface: Giao diện lập trình ứng dụng, dùng cho các dịch vụ RESTful.
RESTful	Representational State Transfer: Phong cách kiến trúc cho API, dùng để xử lý các yêu cầu từ người dùng.
IDE	Integrated Development Environment: Môi trường phát triển tích hợp (ví dụ: IntelliJ, Eclipse).
pom.xml	Project Object Model (XML): Tệp cấu hình dự án Maven, quản lý các dependency.
SQL	Structured Query Language: Ngôn ngữ truy vấn có cấu trúc, được sử dụng để tương tác với SQL Server.
PK	Primary Key: Khóa chính, ràng buộc cơ bản cho các bảng dữ liệu.
FK	Foreign Key: Khóa ngoại, ràng buộc liên kết giữa các bảng.
JDBC	Java Database Connectivity: API kết nối Java với cơ sở dữ liệu SQL Server.
JPA	Java Persistence API: Tiêu chuẩn Java để quản lý dữ liệu quan hệ, đơn giản hóa tương tác DB.
Hibernate	(Tên Framework): Triển khai phổ biến của JPA, được sử dụng trong ứng dụng.
UI	User Interface: Giao diện người dùng, được xây dựng bằng Thymeleaf và HTML/CSS.
UX	User Experience: Trải nghiệm người dùng, hướng tới sự tiện lợi và cá nhân hóa.
JWT	JSON Web Token: Tiêu chuẩn xác thực và ủy quyền, được lưu trữ trong localStorage để duy trì phiên đăng nhập.

RFC	Request for Comments: Tài liệu tiêu chuẩn kỹ thuật (ví dụ: RFC 7519 quy định về JWT).
HMAC	Hash-based Message Authentication Code: Thuật toán ký mã xác thực thông điệp (dùng trong JWT).
RSA	Rivest–Shamir–Adleman: Thuật toán mã hóa khóa công khai (dùng trong JWT).
KNN	K-Nearest Neighbors: Thuật toán đề xuất được đề cập trong Collaborative Filtering.
SVD	Singular Value Decomposition: Thuật toán phân tích ma trận trong hệ thống đề xuất.
CDN	Content Delivery Network: Mạng lưới phân phối nội dung, được đề xuất để cải thiện hiệu suất trong tương lai.
MVC	Model-View-Controller: Mô hình kiến trúc đa tầng mà mã nguồn được tổ chức theo.
CSDL	Cơ sở dữ liệu: Hệ thống lưu trữ dữ liệu (SQL Server) của ứng dụng.
ERD	Entity-Relationship Diagram: Sơ đồ Quan hệ Thực thể, mô hình thiết kế CSDL.
VOD	Video On Demand: Video theo yêu cầu (ngữ cảnh HLS, dù đây là ứng dụng audio).
DL	Deep Learning: Học sâu, được đề xuất để nâng cấp thuật toán gợi ý trong tương lai.
ML	Machine Learning: Học máy, được đề xuất để nâng cấp thuật toán gợi ý.
iOS	iPhone Operating System: Hệ điều hành di động của Apple, hướng phát triển ứng dụng.
.ts / .m3u8	(Định dạng file HLS): Tập phân đoạn media và tập playlist index (HLS).
UUID	Universally Unique Identifier: Mã định danh duy nhất toàn cục.

DANH MỤC HÌNH ẢNH

Hình 2.1: Mô hình Use Case tổng quát	18
Hình 2.2: Use case phân rã chức năng quản lý người dùng	19
Hình 2.3: Use case phân rã chức năng quản lý nhạc	19
Hình 2.4: Use case phân rã chức năng quản lý danh sách nhạc	20
Hình 2.5: Use case phân rã chức năng quản lý hệ thống.....	20
Hình 2.6: Class Diagram	21
Hình 3. 1 Mô hình ERD.....	23
Hình 3. 2 Giao diện đăng kí.....	25
Hình 3. 3 Giao diện đăng nhập	25
Hình 3. 4 Giao diện User.....	26
Hình 3. 5 Giao diện Admin.....	26
Hình 3. 6 Giao diện quản lý User (Admin)	27
Hình 3. 7 Giao diện quản lý Nhạc (Admin)	27
Hình 3. 8 Thư mục lưu trữ các tệp âm thanh đã được xử lý.....	28

DANH MỤC BẢNG

Bảng 2. 1 Dữ liệu bảng User	12
Bảng 2. 2 Dữ liệu bảng Favorite	13
Bảng 2. 3 Dữ liệu bảng Music	13
Bảng 2. 4 Dữ liệu bảng Playlist.....	14
Bảng 2. 5 Dữ liệu bảng Playlist_Item	14
Bảng 2. 6 Dữ liệu bảng Listening_History.....	15

CHƯƠNG 1: TỔNG QUÁT

1.1 Giới thiệu

Trong bối cảnh công nghệ số đang phát triển vượt bậc, nhu cầu giải trí của con người, đặc biệt trong lĩnh vực âm nhạc, ngày càng gia tăng. Các dịch vụ phát nhạc trực tuyến như Spotify, Apple Music hay Zing MP3 đã trở thành một phần quen thuộc trong cuộc sống, cho phép người dùng thưởng thức âm nhạc mọi lúc, mọi nơi. Tuy nhiên, không phải ai cũng có đủ thời gian để tự tìm kiếm và chọn lọc bài hát phù hợp với sở thích cá nhân. Do đó, việc phát triển một ứng dụng phát nhạc trực tuyến tích hợp khả năng đề xuất bài hát tự động dựa trên thói quen nghe nhạc của người dùng trở thành một hướng đi quan trọng, nhằm tối ưu hóa trải nghiệm và nâng cao sự tiện lợi cho người nghe.

Đề tài “Ứng dụng phát trực tuyến nhạc với đề xuất tự động dựa trên sở thích người dùng” ra đời để đáp ứng nhu cầu này, kết hợp giữa công nghệ phát trực tuyến tiên tiến và các thuật toán gợi ý thông minh. Hệ thống áp dụng giao thức HLS để đảm bảo chất lượng phát nhạc ổn định với độ trễ thấp, đồng thời tự điều chỉnh theo điều kiện mạng. Các thuật toán gợi ý cơ bản được triển khai để phân tích dữ liệu lịch sử nghe nhạc, từ đó đề xuất bài hát phù hợp. Người dùng có thể tạo danh sách phát tùy chỉnh, tìm kiếm nội dung yêu thích và nhận gợi ý qua giao diện dễ sử dụng.

1.2 Nhiệm vụ

1.2.1 Tính cấp thiết và lý do chọn đề tài

Trong thời đại số hóa hiện nay, nhu cầu giải trí qua âm nhạc không ngừng gia tăng, đặc biệt với các nền tảng phát trực tuyến. Tuy nhiên, nhiều ứng dụng hiện có vẫn tồn tại những hạn chế trong việc cá nhân hóa trải nghiệm nghe nhạc. Các nền tảng quốc tế thường yêu cầu người dùng trả phí để truy cập đầy đủ tính năng, trong khi các ứng dụng trong nước thường chỉ đề xuất dựa trên xu hướng chung, chưa khai thác sâu sở thích cá nhân. Điều này khiến người dùng tốn thời gian để tự tìm kiếm bài hát phù hợp, làm giảm sự thoải mái khi sử dụng.

Xuất phát từ thực trạng trong việc phát triển một ứng dụng phát nhạc trực tuyến với tính năng gợi ý tự động trở nên cần thiết, giúp người dùng tiếp cận nội dung phù hợp mà không cần nỗ lực nhiều. Hệ thống không chỉ cung cấp phát nhạc mượt mà

mà còn phân tích sở thích cá nhân, mang lại trải nghiệm nghe nhạc thông minh. Việc chọn Java làm nền tảng đảm bảo tính ổn định, khả năng hoạt động trên nhiều thiết bị và tiềm năng mở rộng trong tương lai.

Với xu hướng công nghệ giải trí phát triển, đề tài được lựa chọn để thiết kế và triển khai một hệ thống phát nhạc trực tuyến, đáp ứng các yêu cầu phát trực tuyến, gợi ý thông minh và quản lý danh sách phát. Hệ thống góp phần cải thiện trải nghiệm người dùng, tăng cường tương tác và phù hợp với định hướng giải trí hiện đại tập trung vào cá nhân hóa.

1.2.2 Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là xây dựng một ứng dụng phát nhạc trực tuyến sử dụng Java, hỗ trợ phát nhạc mượt mà và cung cấp gợi ý bài hát tự động dựa trên sở thích cá nhân, mang lại trải nghiệm giải trí tiện lợi và hiệu quả. Hệ thống được thiết kế linh hoạt, dễ sử dụng, giúp người dùng thưởng thức âm nhạc theo cách cá nhân hóa, đồng thời đóng góp vào việc nâng cao chất lượng giải trí số.

Để đạt được mục tiêu này, đề tài tập trung vào việc phát triển hệ thống quản lý người dùng với các tính năng đăng ký, đăng nhập và quản lý thông tin cá nhân, đảm bảo sự thuận tiện và an toàn cơ bản. Người dùng có thể cập nhật thông tin, thay đổi mật khẩu và theo dõi lịch sử nghe nhạc một cách dễ dàng.

Một trọng tâm khác là xây dựng các chức năng phát nhạc trực tuyến và gợi ý thông minh. Hệ thống sử dụng giao thức HLS để phát nhạc với độ trễ thấp, tự động điều chỉnh chất lượng theo băng thông mạng. Đồng thời, các thuật toán gợi ý đơn giản được tích hợp để phân tích dữ liệu lịch sử nghe, đề xuất bài hát phù hợp và hiển thị kết quả trực quan. Hệ thống cũng hỗ trợ tạo danh sách phát và tìm kiếm nội dung.

Cuối cùng, hệ thống đảm bảo hiệu suất và bảo mật thông qua việc lưu trữ dữ liệu trên SQL Server với cơ chế mã hóa cơ bản, phòng tránh các lỗ hổng như SQL Injection. Sử dụng JDBC để kết nối với SQL Server, hệ thống tối ưu hóa truy vấn và duy trì tính toàn vẹn dữ liệu. Hiệu năng được đảm bảo để xử lý nhiều người dùng đồng thời, với các biện pháp sao lưu và khôi phục nhằm đảm bảo hoạt động liên tục.

Tổng thể, đề tài hướng đến việc tạo ra một ứng dụng phát nhạc trực tuyến toàn diện, hiệu quả, đáp ứng nhu cầu thực tế và góp phần thúc đẩy ứng dụng công nghệ trong lĩnh vực âm nhạc số.

1.2.3 Đối tượng và phạm vi nghiên cứu

Đề tài tập trung nghiên cứu phục vụ người dùng phổ thông có nhu cầu nghe nhạc trực tuyến, đặc biệt là những người mong muốn thưởng thức âm nhạc và nhận gợi ý bài hát dựa trên sở thích cá nhân, cùng với quản trị viên để quản lý nội dung bài hát và tài khoản người dùng.

Về chức năng, hệ thống được thiết kế để hỗ trợ các nghiệp vụ cốt lõi như phát nhạc trực tuyến thông qua giao thức HLS, gợi ý bài hát tự động dựa trên lịch sử nghe nhạc, quản lý danh sách bài hát, tìm kiếm bài hát, và quản lý tài khoản người dùng bao gồm đăng ký, đăng nhập, cập nhật thông tin.

Về triển khai, đề tài tập trung vào thiết kế, phát triển, và kiểm thử hệ thống trên môi trường thực tế, đảm bảo hiệu suất và tính ổn định, với cơ sở dữ liệu được xây dựng hợp lý trong SQL Server để hỗ trợ quản trị dữ liệu hiệu quả.

Các công cụ sử dụng bao gồm Java làm nền tảng chính cho backend để xử lý logic nghiệp vụ, HTML/CSS để xây dựng giao diện web đơn giản và thân thiện, SQL Server để lưu trữ thông tin người dùng, bài hát, danh sách phát và lịch sử nghe nhạc, JDBC để kết nối Java với SQL Server nhằm thực hiện truy vấn dữ liệu hiệu quả, và HLS để truyền tải âm thanh mượt mà với chất lượng thích ứng theo tốc độ mạng.

1.3 Cấu trúc

Chương 1. Tổng quan: Chương này giới thiệu khái quát về đề tài, trình bày bối cảnh hình thành, tính cấp thiết, mục tiêu nghiên cứu, cũng như lý do lựa chọn đề tài. Đồng thời, chương làm rõ nhiệm vụ nghiên cứu, đối tượng, phạm vi nghiên cứu, công cụ sử dụng, và mô tả cấu trúc tổng thể của báo cáo nhằm giúp người đọc dễ dàng theo dõi các nội dung chính.

Chương 2. Cơ sở lý thuyết: Chương này trình bày các kiến thức nền tảng và lý thuyết liên quan, là cơ sở khoa học cho việc phát triển hệ thống. Nội dung bao gồm tổng quan về công nghệ phát trực tuyến HLS, hệ thống gợi ý, lập trình Java, và cơ sở dữ liệu SQL

Server. Ngoài ra, chương cũng giới thiệu các công cụ như HTML/CSS, JDBC, và phương pháp phát triển phần mềm được áp dụng.

Chương 3. Kết quả thực nghiệm: Chương này tập trung trình bày quá trình phân tích và thiết kế hệ thống, từ yêu cầu người dùng đến các sơ đồ chức năng và dữ liệu. Sau đó là phần mô tả chi tiết việc cài đặt, triển khai hệ thống bằng Java, tích hợp HTML/CSS và SQL Server. Kết thúc chương là phần trình bày các kết quả đạt được, minh họa qua các tính năng như phát nhạc trực tuyến, gợi ý bài hát, và quản lý danh sách phát.

Chương 4. Kết luận và kiến nghị: Chương cuối cùng tóm tắt những nội dung đã thực hiện trong đồ án, đánh giá hiệu quả của hệ thống sau khi triển khai. Bên cạnh đó, chương cũng nêu lên những mặt còn hạn chế, từ đó đưa ra các đề xuất và định hướng phát triển trong tương lai nhằm nâng cao tính ứng dụng và hoàn thiện hệ thống hơn nữa.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Nền tảng & công nghệ

2.1.1 *Spring Boot 3, Java 17 và Maven*

Spring Boot là một framework mã nguồn mở dựa trên Java, được thiết kế để xây dựng các ứng dụng web và microservices một cách nhanh chóng và hiệu quả. Phiên bản Spring Boot 3, kết hợp với Java 17, mang lại nhiều cải tiến đáng kể. Theo tài liệu từ Spring.io, Spring Boot 3 hỗ trợ đầy đủ Java 17+, bao gồm các tính năng như Pattern Matching cho switch, Sealed Classes, và Records giảm boilerplate code cho các lớp dữ liệu. Những tính năng này giúp cải thiện hiệu suất và bảo trì mã nguồn.

Java 17 là phiên bản LTS, cung cấp các cải tiến về hiệu suất như giảm thời gian khởi động JVM và quản lý bộ nhớ hiệu quả hơn, theo bài viết từ Atlantbh. Điều này rất quan trọng cho ứng dụng streaming nhạc, nơi cần xử lý nhanh chóng các yêu cầu từ người dùng.

Maven là công cụ quản lý dự án, sử dụng file pom.xml để định nghĩa dependencies. Trong đồ án, chúng ta sử dụng các starter như spring-boot-starter-web cho API RESTful, spring-boot-starter-security, và spring-boot-starter-data-jpa. Maven tự động hóa quá trình build, test và deploy, đảm bảo tính nhất quán.

Quá trình tổ chức project với Maven bao gồm cấu trúc thư mục chuẩn: src/main/java cho code, src/main/resources cho configuration như application.properties, và pom.xml để quản lý versions. Spring Boot 3 còn hỗ trợ native compilation với GraalVM, giúp giảm thời gian khởi động và kích thước image Docker, theo bài viết từ Medium về các tính năng mới.

2.1.2 *SQL Server*

SQL Server là một hệ quản trị cơ sở dữ liệu mạnh mẽ, được sử dụng rộng rãi trong quản lý và lưu trữ dữ liệu. Nó không chỉ hỗ trợ các tính năng cơ bản mà còn cung cấp nhiều công cụ nâng cao giúp người dùng tối ưu hóa việc quản lý dữ liệu.

SQL Server hỗ trợ các chức năng thiết yếu như thiết kế bảng với các trường dữ liệu linh hoạt, tạo chỉ mục indexing để tăng tốc truy vấn, áp dụng ràng buộc constraints như khóa chính, khóa ngoại và kiểm tra duy nhất, cũng như sử dụng

triggers để tự động thực thi các hành động khi dữ liệu thay đổi. Về bảo mật, SQL Server cung cấp các cơ chế toàn diện bao gồm authentication, authorization, encryption và các công cụ kiểm toán auditing để ghi lại hoạt động truy cập.

Để nâng cao hiệu suất, SQL Server tích hợp các kỹ thuật tối ưu hóa truy vấn query optimization, quản lý chỉ mục index management, thống kê statistics để phân tích dữ liệu, và bộ nhớ đệm để giảm thời gian truy xuất.

Cuối cùng, SQL Server cung cấp các công cụ bảo trì mạnh mẽ như backup, restore, maintenance plans, và monitoring. Những công cụ này đảm bảo cơ sở dữ liệu hoạt động ổn định và an toàn trong môi trường sản xuất, giảm thiểu rủi ro mất dữ liệu và đảm bảo hệ thống luôn sẵn sàng phục vụ người dùng, đặc biệt trong trường hợp cần khôi phục dữ liệu lịch sử nghe nhạc sau sự cố.

2.1.3 JPA/Hibernate

JPA là một tiêu chuẩn Java dành cho quản lý dữ liệu quan hệ, cung cấp một giao diện thống nhất để ánh xạ các đối tượng Java entities với các bảng trong cơ sở dữ liệu, giúp giảm thiểu code boilerplate và tăng tính tái sử dụng. Hibernate, một trong những triển khai phổ biến nhất của JPA, được sử dụng trong đồ án để đơn giản hóa quá trình tương tác với SQL Server, đặc biệt trong việc quản lý các thực thể như người dùng, bài hát, và lịch sử nghe nhạc. Các entity chính trong hệ thống:

Bảng 2. 1 Dữ liệu bảng User

STT	Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
1	id	INT	Mã định danh người dùng	PK
2	created_at	DATETIME	Thời gian tạo tài khoản	NOT NULL
3	email	VARCHAR(255)	Địa chỉ email người dùng	NOT NULL, UNIQUE
4	password	VARCHAR(255)	Mật khẩu (đã mã hóa)	NOT NULL
5	role	VARCHAR(50)	Vai trò của người dùng	NOT NULL
6	username	VARCHAR(50)	Tên người dùng	NOT NULL, UNIQUE

Bảng users quản lý thông tin cơ bản về người dùng trên nền tảng. Mỗi bản ghi bao gồm mã định danh người dùng là khóa chính, cùng với thông tin bắt buộc như thời gian tạo, email, password, role và username yêu cầu không được để trống. Thuộc tính email và username cũng được yêu cầu phải là duy nhất.

Bảng 2. 2 Dữ liệu bảng Favorite

STT	Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
1	user_id	INT	Mã người dùng	PK, FK → users(id)
2	music_id	INT	Mã bài hát yêu thích	PK, FK→music(id)

Bảng favorites ghi lại mối quan hệ giữa người dùng và các bài hát họ yêu thích. Bảng này sử dụng một khóa chính kép bao gồm user_id và music_id. Cả hai thuộc tính này đều là khóa ngoại, liên kết đến bảng users và music để đảm bảo tính toàn vẹn của dữ liệu về bài hát yêu thích

Bảng 2. 3 Dữ liệu bảng Music

STT	Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
1	id	INT	Mã định danh bài hát	PK
2	album	NVARCHAR(255)	Tên album	-
3	artist	NVARCHAR(255)	Tên nghệ sĩ	NOT NULL
4	cover_url	VARCHAR(500)	Đường dẫn dự phòng	-
5	created_at	DATETIME	Thời gian thêm bài hát	NOT NULL
6	genre	NVARCHAR(100)	Thể loại nhạc	-
7	title	NVARCHAR(255)	Tiêu đề bài hát	NOT NULL
8	url	VARCHAR(500)	Đường dẫn file nhạc	NOT NULL
9	description	NVARCHAR(MAX)	Mô tả chi tiết bài hát	-

Bảng music lưu trữ thông tin chi tiết về các bài hát trong hệ thống. Mã định danh bài hát là khóa chính. Các thuộc tính bắt buộc gồm có artist, created_at, title và url yêu cầu không được để trống. Các thuộc tính còn lại như album, cover_url, genre và description có thể để trống.

Bảng 2. 4 Dữ liệu bảng Playlist

STT	Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
1	id	INT	Mã định danh Playlist	PK
2	created_at	DATETIME	Thời gian tạo Playlist	NOT NULL
3	name	NVARCHAR(255)	Tên Playlist	NOT NULL
4	user_id	INT	Mã người sở hữu Playlist	FK → users(id), NOT NULL

Playlist quản lý các danh sách phát nhạc do người dùng tạo. Mã định danh Playlist là khóa chính. Các thuộc tính created_at, name, và user_id yêu cầu không được để trống. Thuộc tính user_id là khóa ngoại liên kết đến bảng users để xác định chủ sở hữu.

Bảng 2. 5 Dữ liệu bảng Playlist_Item

STT	Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
1	playlist_id	INT	Mã Playlist	PK, FK → playlists(id)
2	music_id	INT	Mã bài hát trong Playlist	PK, FK → music(id)

Bảng playlist_items xác định các bài hát có trong mỗi Playlist. Đây là một bảng trung gian sử dụng khóa chính kép bao gồm playlist_id và music_id. Cả hai thuộc tính đều là khóa ngoại, liên kết đến bảng playlists và music, đảm bảo các bài hát được thêm vào là hợp lệ và thuộc về một playlist tồn tại.

Bảng 2. 6 Dữ liệu bảng *Listening_History*

STT	Thuộc tính	Kiểu dữ liệu	Mô tả	Ràng buộc
1	id	INT	Mã định danh lịch sử nghe	PK
2	played_at	DATETIME	Thời điểm bài hát được nghe	NOT NULL
3	music_id	INT	Mã bài hát đã nghe	FK → music(id), NOT NULL
4	user_id	INT	Mã người dùng đã nghe	FK → users(id), NOT NULL

Listening_history theo dõi lịch sử phát nhạc của người dùng. Mã định danh lịch sử là khóa chính. Các thuộc tính *played_at*, *music_id* và *user_id* yêu cầu không được để trống. Thuộc tính *music_id* và *user_id* là các khóa ngoại liên kết đến bảng *music* và *users*.

2.1.4 *Thymeleaf*

Trong lĩnh vực phát triển ứng dụng web, việc xây dựng giao diện người dùng UI thân thiện và hiệu quả là một yếu tố quan trọng, đặc biệt với các ứng dụng đòi hỏi tương tác động như streaming nhạc.

Thymeleaf là một engine template mạnh mẽ cho Java, đáp ứng nhu cầu này bằng cách tích hợp chặt chẽ với Spring Boot để render các trang HTML động, hỗ trợ các biểu thức như *th:if*, *th:each*, và *th:object* để binding dữ liệu từ model sang giao diện.

Thymeleaf được sử dụng để xây dựng các giao diện chính: *home.html* hiển thị danh sách nhạc và chức năng tìm kiếm, *auth/login.html* cùng *auth/register.html* cung cấp form đăng nhập và đăng ký với lưu token trong *localStorage*, và *admin/dashboard.html* phục vụ bảng điều khiển với thống kê từ lịch sử nghe.

Theo tài liệu từ Thymeleaf.org, tích hợp với Spring MVC được thực hiện qua *ThymeleafViewResolver*, kích hoạt tự động với dependency *spring-boot-starter-*

thymeleaf, đồng thời tính năng natural template cho phép xem template như HTML tĩnh trong trình duyệt, tăng tốc phát triển UI/UX.

2.1.5 HTTP Live Streaming (HLS)

Trong lĩnh vực phát trực tuyến, việc cung cấp trải nghiệm mượt mà và chất lượng cao cho người dùng là một thách thức lớn, đặc biệt khi băng thông mạng thay đổi liên tục. HLS là một giao thức streaming do Apple phát triển, ra đời để giải quyết vấn đề này với khả năng điều chỉnh chất lượng dựa trên băng thông.

HLS hoạt động dựa trên nguyên lý phân mảnh file media thành các segments .ts, đồng thời tạo playlist .m3u8 chứa danh sách các segments cùng metadata bitrate 128k, 256k. Adaptive bitrate cho phép client chuyển đổi mượt mà giữa các chất lượng dựa trên điều kiện mạng.

HlsService sử dụng ffmpeg để transcode file âm thanh: `ffmpeg -i input.mp3 -f hls -hls_time 10 master.m3u8`. Theo Mux.com, ffmpeg hỗ trợ tạo multiple variants cho adaptive streaming, tối ưu hóa trải nghiệm người dùng.

Player client-side sử dụng HLS.js để parse file .m3u8 và phát lại trong thẻ. HLS giúp giảm thiểu buffering và hỗ trợ cả streaming live lẫn on-demand, phù hợp với yêu cầu của hệ thống.

2.1.6 JSON Web Token và bảo mật

Trong lĩnh vực phát triển ứng dụng web, việc đảm bảo an toàn thông tin và xác thực danh tính người dùng là một yêu cầu thiết yếu, đặc biệt đối với các hệ thống đòi hỏi bảo mật cao như ứng dụng phát trực tuyến nhạc với các tính năng cá nhân hóa. JSON Web Token, một tiêu chuẩn được quy định trong RFC 7519, nổi bật như một giải pháp hiệu quả để truyền tải thông tin an toàn giữa các thành phần của hệ thống.

JWT được cấu trúc với ba phần chính: Header chứa thông tin thuật toán và loại token, Payload mang dữ liệu như mã người dùng, vai trò và thời gian hết hạn, cùng Signature đảm bảo tính toàn vẹn và xác thực thông qua ký bằng HMAC hoặc RSA.

JWT được triển khai để quản lý quá trình xác thực và ủy quyền. JwtUtil đảm nhận việc tạo và xác minh token, trong khi các middleware như JwtAuthenticationFilter

và WebJwtAuthenticationFilter được sử dụng để kiểm tra token trong các yêu cầu API và giao diện người dùng, bao gồm hỗ trợ tham số truy vấn tạm thời.

Bảo mật tính được thực hiện thông qua SecurityConfig, nơi sử dụng HttpSecurity để định nghĩa các tuyến đường công khai và các tuyến đường yêu cầu vai trò ADMIN hỗ trợ phân quyền dựa trên vai trò theo tài liệu chính thức của Spring.

2.1.7 Phương pháp đề xuất tự động

Trong lĩnh vực công nghệ thông tin, đặc biệt là các nền tảng trực tuyến như phát trực tuyến âm nhạc, việc cung cấp gợi ý cá nhân hóa đã trở thành một yếu tố quan trọng để nâng cao trải nghiệm người dùng và giữ chân khách hàng. Hệ thống đề xuất Recommendation System tận dụng dữ liệu hành vi và sở thích để đưa ra các gợi ý phù hợp, giúp người dùng khám phá nội dung mới một cách hiệu quả. Trong bối cảnh đó, các phương pháp khác nhau được phát triển để tối ưu hóa quá trình này, từ việc phân tích nội dung đến việc học hỏi từ cộng đồng người dùng.

Hệ thống đề xuất được xây dựng dựa trên dữ liệu từ Listening History để cá nhân hóa trải nghiệm. Các phương pháp được áp dụng bao gồm: Content-based Filtering, sử dụng cosine similarity để so sánh các vectors dựa trên thuộc tính như genre và artist; Collaborative Filtering, dựa trên hành vi của người dùng tương tự thông qua các kỹ thuật như KNN hoặc SVD theo hướng user-based hoặc item-based; và Hybrid, kết hợp cả hai phương pháp để khắc phục vấn đề cold-start.

Theo nghiên cứu từ ScienceDirect, phương pháp Hybrid có thể cải thiện độ chính xác lên 20-30%, mang lại hiệu quả cao hơn trong việc đề xuất nhạc phù hợp.

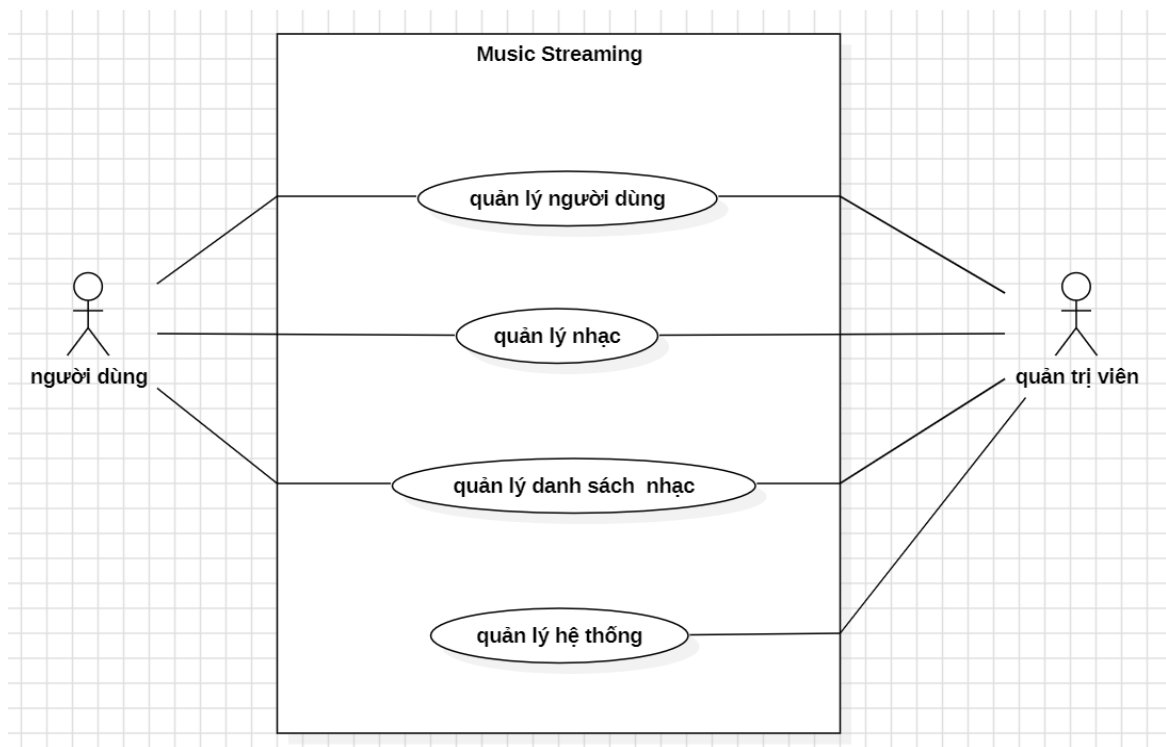
2.2 Kiến trúc

Trong thiết kế hệ thống phần mềm hiện đại, việc xây dựng một kiến trúc rõ ràng và phân tách chức năng là yếu tố then chốt để đảm bảo khả năng mở rộng, bảo trì và hiệu suất cao, đặc biệt với một ứng dụng phức tạp như phát trực tuyến nhạc. Hệ thống "Ứng dụng phát trực tuyến nhạc với đề xuất tự động dựa trên sở thích người dùng" được thiết kế theo mô hình kiến trúc nhiều lớp, phân chia các thành phần chức năng một cách logic. Hệ thống bao gồm các lớp sau:

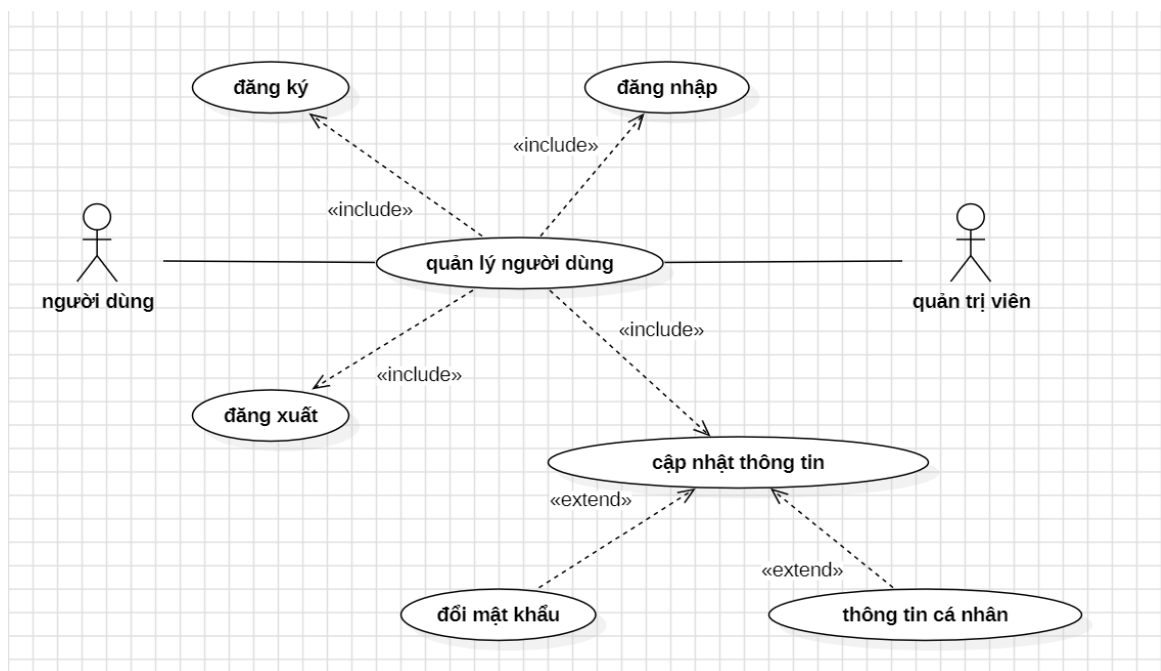
- Controller Layer: Chịu trách nhiệm xử lý các yêu cầu HTTP từ người dùng, đóng vai trò trung gian giữa giao diện và logic nghiệp vụ.
- Service Layer: Chứa các logic nghiệp vụ chính, bao gồm MusicService quản lý bài hát, HlsService xử lý streaming, và RecommendationService tính toán đề xuất dựa trên sở thích.
- Repository Layer: Thực hiện truy xuất dữ liệu từ cơ sở dữ liệu thông qua JpaRepository, đảm bảo tương tác hiệu quả với SQL Server.
- Model Layer: Bao gồm các entity như User, Music, Playlist, và ListeningHistory, đại diện cho cấu trúc dữ liệu của hệ thống.
- Config Layer: Quản lý các cấu hình hệ thống, bao gồm SecurityConfig bảo mật và StaticResourceConfig tài nguyên tĩnh.
- Security Layer: Sử dụng các filter JWT để xác thực và ủy quyền người dùng.
- View Layer: Sử dụng Thymeleaf templates để render giao diện động cho người dùng.

2.3 Thiết kế hệ thống

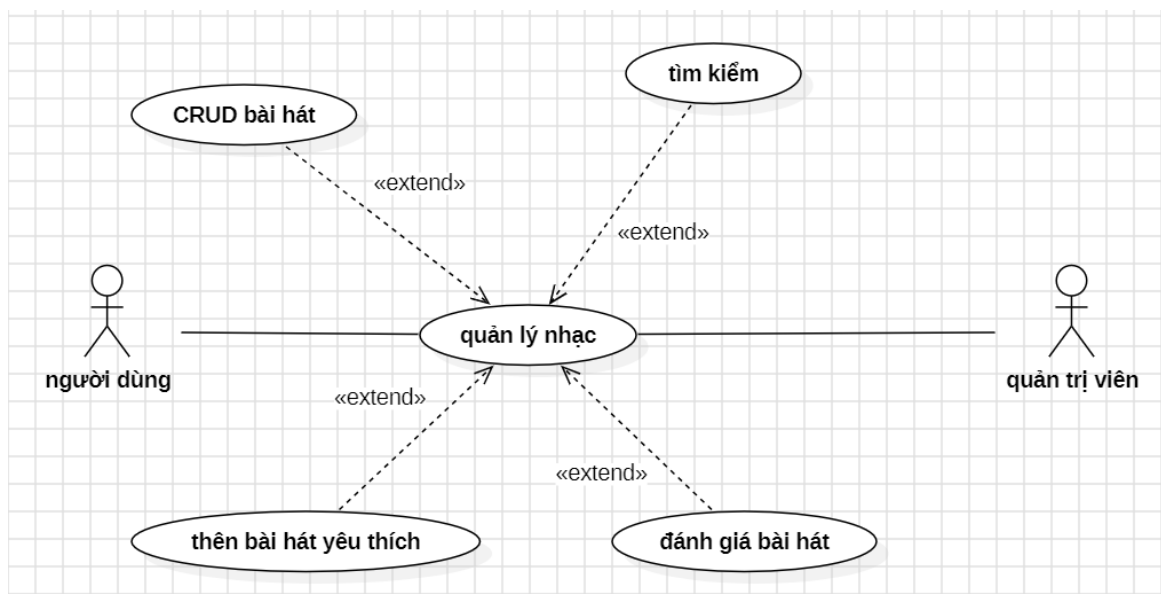
2.3.1 Mô hình Use case Diagram



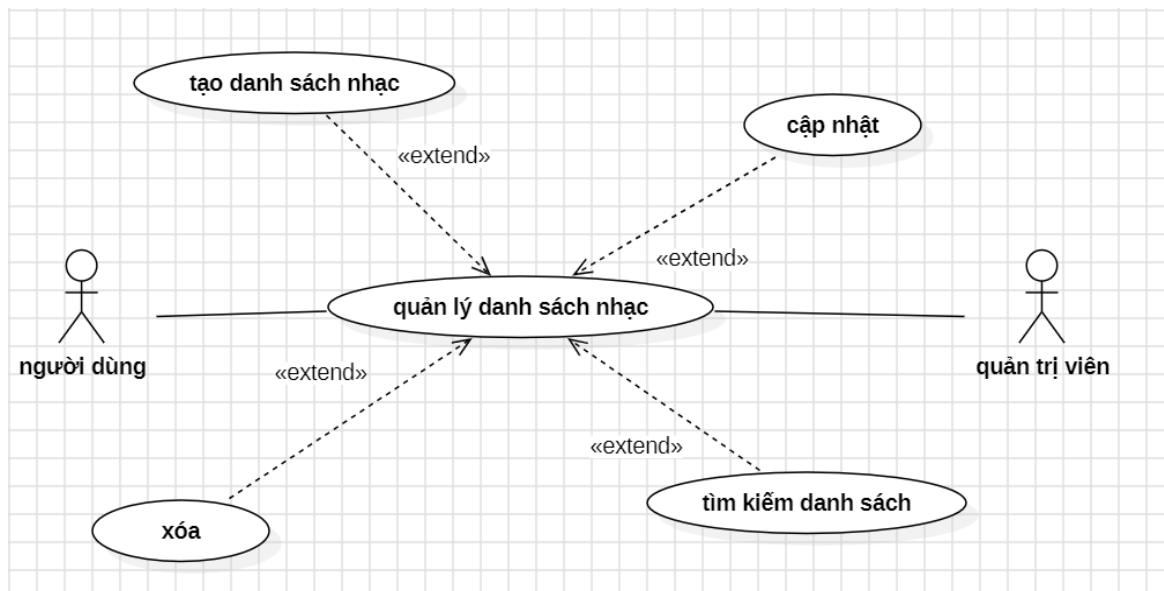
Hình 2.1: Mô hình Use Case tổng quát



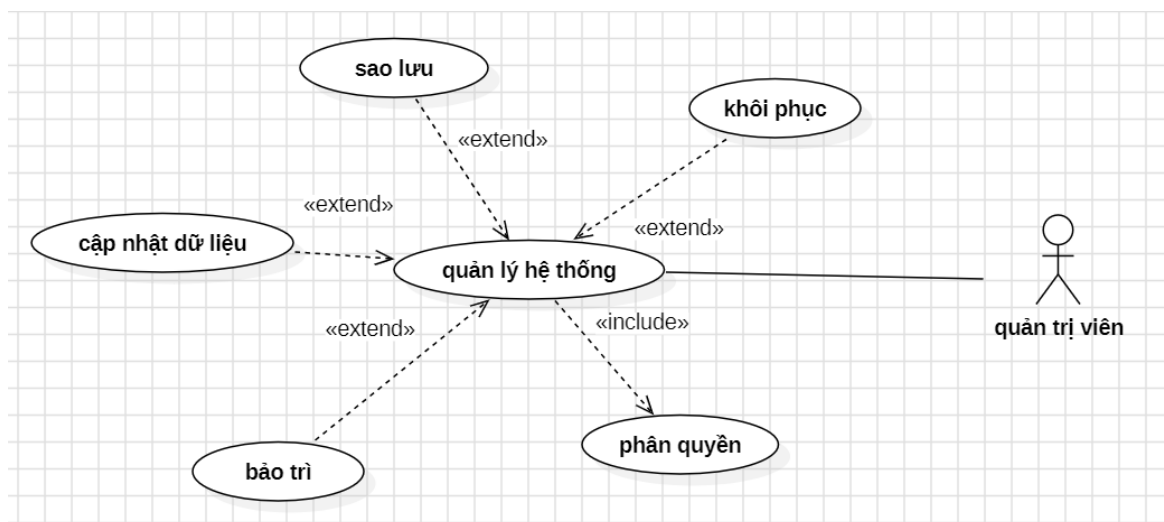
Hình 2.2: Use case phân rã chức năng quản lý người dùng



Hình 2.3: Use case phân rã chức năng quản lý nhạc

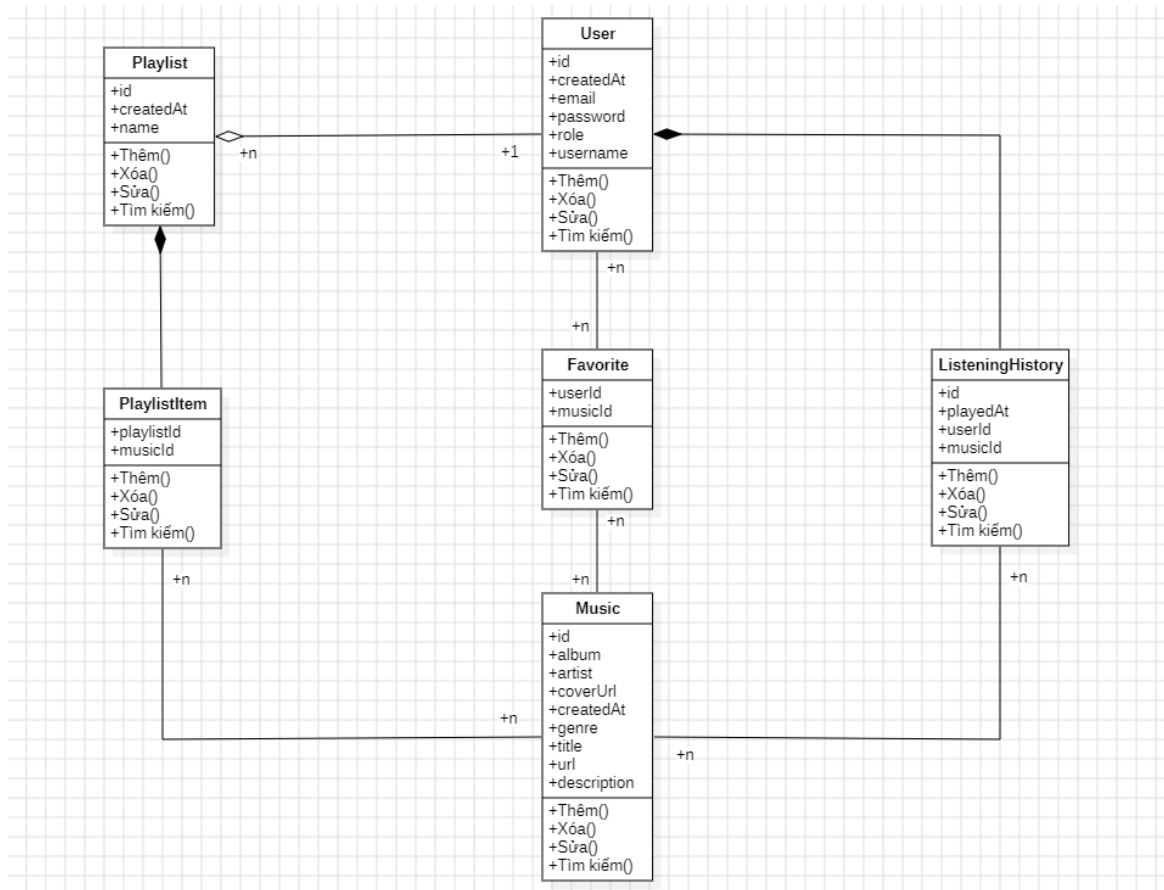


Hình 2.4: Use case phân rã chức năng quản lý danh sách nhạc



Hình 2.5: Use case phân rã chức năng quản lý hệ thống

2.3.2 Class Diagram



Hình 2.6: Class Diagram

CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM

3.1 Triển khai hệ thống

Phần này trình bày chi tiết về quá trình triển khai thực tế hệ thống, từ tổ chức mã nguồn đến thiết kế cơ sở dữ liệu và các chức năng chính đã được hiện thực hóa trong quá trình phát triển.

3.1.1 Cấu hình build/run (Maven, application.properties)

Hệ thống được phát triển bằng Spring Boot và build bằng Maven. File pom.xml chứa các dependency cần thiết như Spring Web, Spring Security, JPA, SQL Server Driver và JWT. Trong file application.properties, hệ thống cấu hình kết nối đến cơ sở dữ liệu SQL Server, đường dẫn lưu trữ file nhạc HLS và các thông số bảo mật. Quá trình chạy hệ thống được thực hiện thông qua lệnh mvn spring-boot:run, giúp tự động biên dịch và khởi động ứng dụng kèm theo server Tomcat tích hợp sẵn.

3.1.2 Tạo CSDL SQL Server, mapping entity

Cơ sở dữ liệu của hệ thống phát nhạc được thiết kế dựa trên mô hình quan hệ, triển khai thông qua Spring Data JPA và SQL Server. Cách tiếp cận theo mô hình Code-First cho phép các lớp Java được ánh xạ trực tiếp thành các bảng trong cơ sở dữ liệu, bảo đảm tính đồng nhất giữa mô hình dữ liệu và logic nghiệp vụ trong ứng dụng.

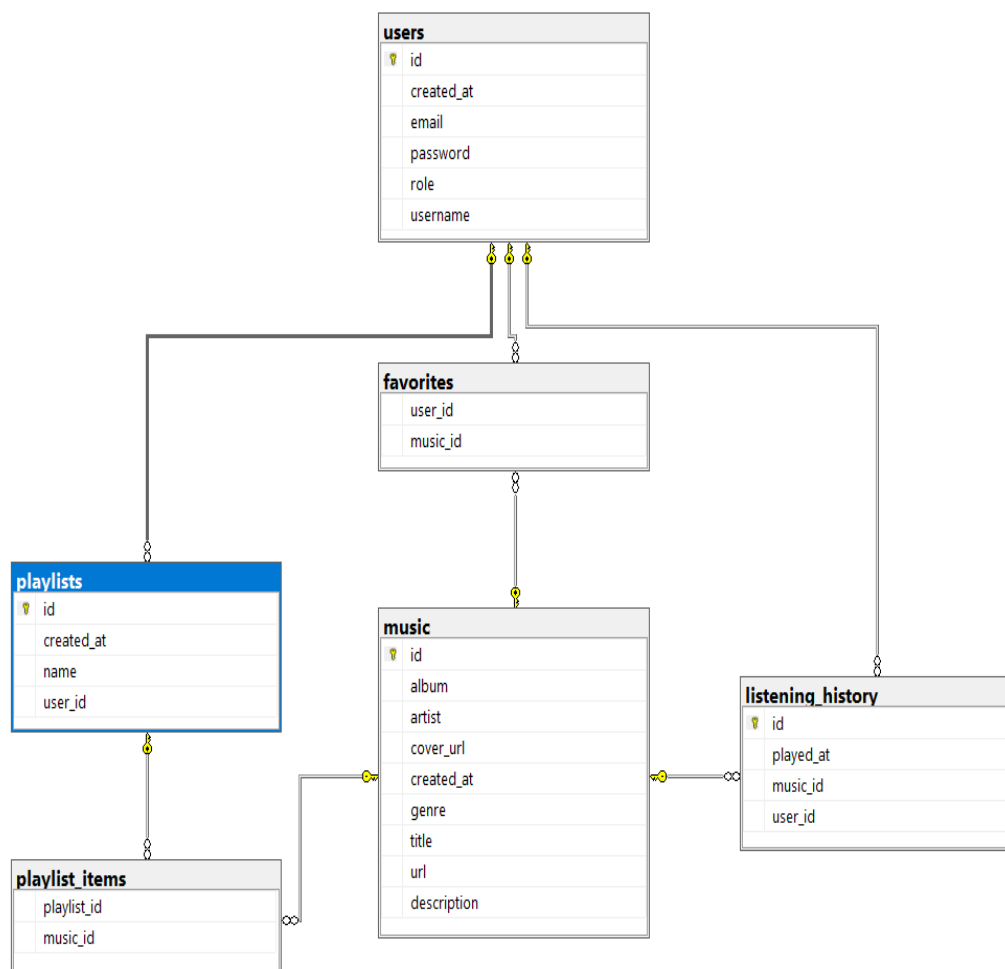
Các bảng dữ liệu chính bao gồm Users, nơi lưu trữ thông tin người dùng như địa chỉ email, mật khẩu được mã hóa, vai trò hệ thống, tên hiển thị và ngày tạo tài khoản. Bảng Music quản lý thông tin chi tiết về các bài hát, gồm tên bài hát, thể loại, nghệ sĩ, album, hình ảnh bìa, đường dẫn phát nhạc HLS, ngày phát hành và mô tả bổ sung.

Bảng Playlists đại diện cho danh sách phát do người dùng tự tạo, gồm tên danh sách, người sở hữu và thời gian khởi tạo. Mỗi danh sách có thể chứa nhiều bài hát, mối quan hệ này được thể hiện qua bảng trung gian Playlist_Items, chứa hai khóa ngoại PlaylistId và MusicId, hình thành liên kết nhiều-nhiều giữa Playlist và Music.

Ngoài ra, bảng Favorites cho phép người dùng đánh dấu các bài hát yêu thích, thể hiện mối quan hệ nhiều-nhiều giữa Users và Music thông qua hai cột UserId và

MusicId. Bảng Listening_History ghi nhận lịch sử nghe nhạc, lưu thông tin người nghe, bài hát đã phát và thời điểm phát. Dữ liệu này được hệ thống sử dụng để phục vụ cho thuật toán gợi ý nhạc theo sở thích cá nhân.

Các mối quan hệ giữa các bảng được định nghĩa rõ ràng thông qua các khóa ngoại và thuộc tính điều hướng trong các lớp thực thể, sử dụng cú pháp như @OneToMany, @ManyToOne hoặc @ManyToMany. Các ràng buộc như @NotNull, @Column và cascade = CascadeType.ALL được áp dụng để bảo đảm tính toàn vẹn và đồng nhất dữ liệu, đồng thời hạn chế lỗi trong quá trình xóa hoặc cập nhật bản ghi.



Hình 3. 1 Mô hình ERD

3.1.3 Chạy *ffmpeg* và luồng xử lý HLS

Bên cạnh việc quản lý dữ liệu, hệ thống còn tích hợp cơ chế xử lý âm thanh tự động thông qua công cụ FFmpeg, được điều khiển trực tiếp bằng lớp HlsService trong Java. Khi người quản trị tải lên một bài hát mới, ứng dụng sẽ gọi phương thức `transcodeToHls()` để thực hiện quá trình chuyển đổi tệp âm thanh gốc thành định dạng HLS, phục vụ cho việc phát nhạc trực tuyến.

Trong quá trình xử lý, hệ thống tự động tạo hai thư mục lưu trữ luồng nhạc có chất lượng khác nhau là 128kbps và 256kbps. Sau khi hoàn tất việc khởi tạo thư mục `hls/{musicId}`, chương trình sử dụng `ProcessBuilder` để khởi chạy hai tiến trình *ffmpeg* riêng biệt, mỗi tiến trình thực hiện chuyển đổi file âm thanh sang định dạng `.ts` với các đoạn có độ dài 6 giây, đồng thời sinh ra file danh sách `index.m3u8`.

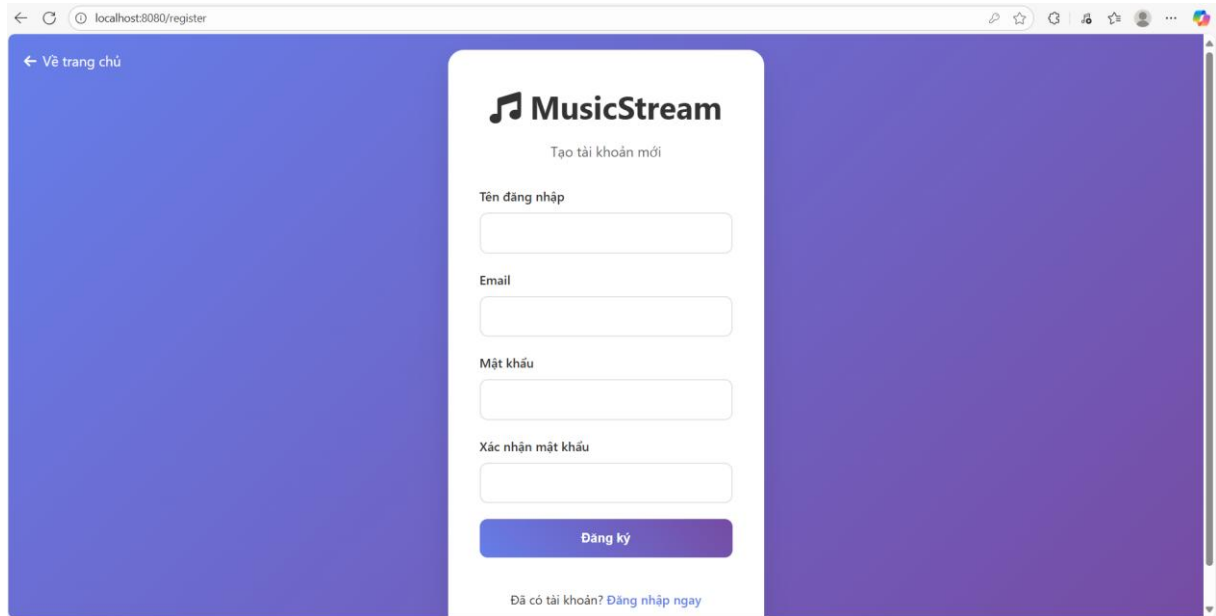
Sau khi hai phiên bản bitrate được tạo thành công, hệ thống tiếp tục sinh ra file `master.m3u8`, đóng vai trò liên kết các danh sách phát con (`128k/index.m3u8` và `256k/index.m3u8`). File này được ghi trực tiếp bằng Java thông qua phương thức `Files.writeString()`. Khi người dùng phát nhạc, trình phát web sử dụng `HLS.js` sẽ tải file `master.m3u8`, phân tích nội dung và phát lần lượt các phân đoạn `.ts` để đảm bảo âm thanh được truyền liên tục và ổn định.

Các file được lưu tại thư mục tĩnh và được công khai qua cấu hình `StaticResourceConfig`, giúp frontend có thể truy cập thông qua HTTP. Cách xử lý này giúp tối ưu hiệu năng phát nhạc, giảm tải cho máy chủ và duy trì chất lượng ổn định trên nhiều loại kết nối mạng.

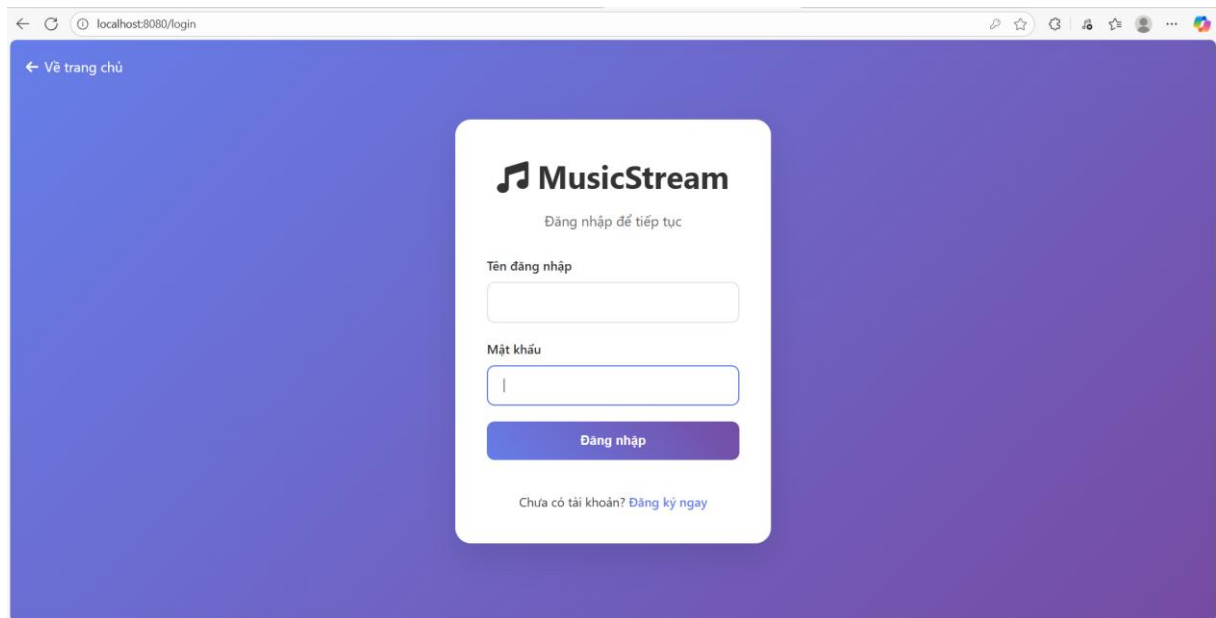
Kết quả kiểm thử cho thấy thời gian chuyển đổi trung bình cho một bài hát từ 5 đến 8 MB chỉ mất khoảng 3–6 giây, và quá trình phát qua HLS đạt hiệu năng tốt, không xảy ra tình trạng gián đoạn. Cơ chế này cũng mở ra khả năng mở rộng trong tương lai như tích hợp CDN, caching hoặc adaptive bitrate streaming để nâng cao trải nghiệm người nghe.

3.2. Tính năng đã hoàn thành

Hệ thống phát nhạc trực tuyến được phát triển đầy đủ các chức năng dành cho hai nhóm người dùng chính là người dùng thông thường và quản trị viên. Người dùng có thể đăng ký và đăng nhập tài khoản, quá trình xác thực được thực hiện bằng JWT và token này được lưu trữ trong localStorage để duy trì phiên đăng nhập.

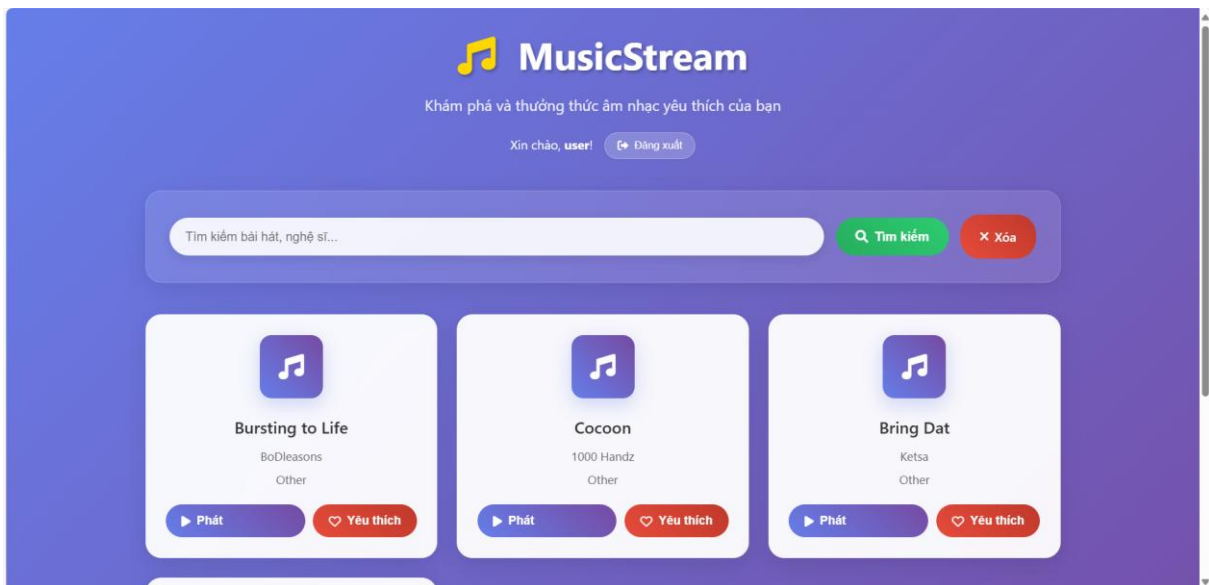


Hình 3. 2 Giao diện đăng kí



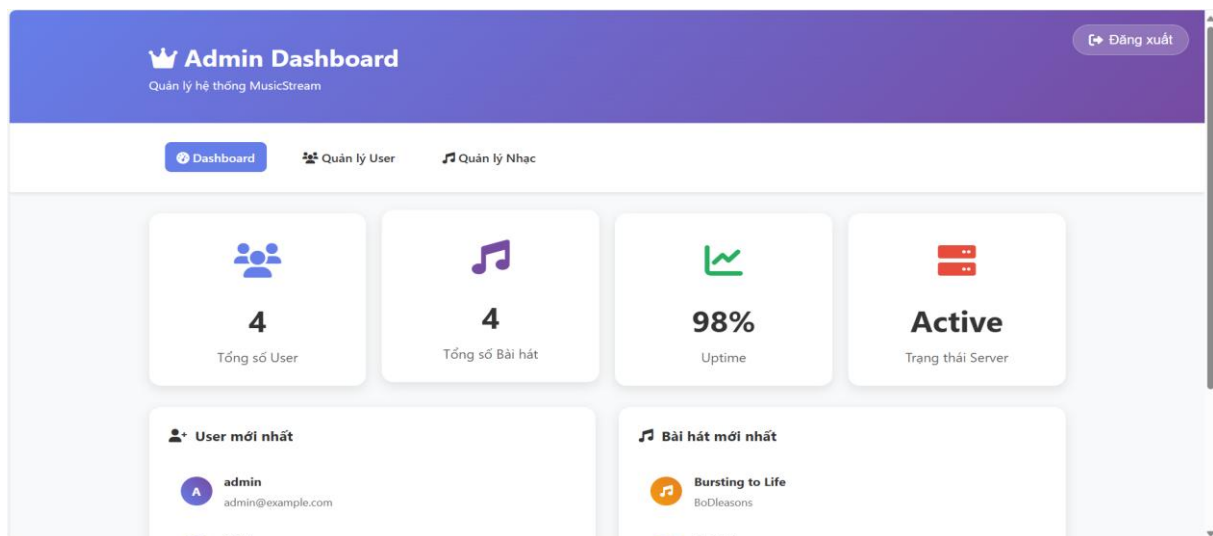
Hình 3. 3 Giao diện đăng nhập

Đối với người dùng, hệ thống cho phép xem danh sách các bài hát, tìm kiếm theo tên hoặc thể loại và phát nhạc trực tiếp thông qua giao thức HLS sử dụng thư viện HLS.js giúp đảm bảo khả năng tương thích trên nhiều thiết bị.

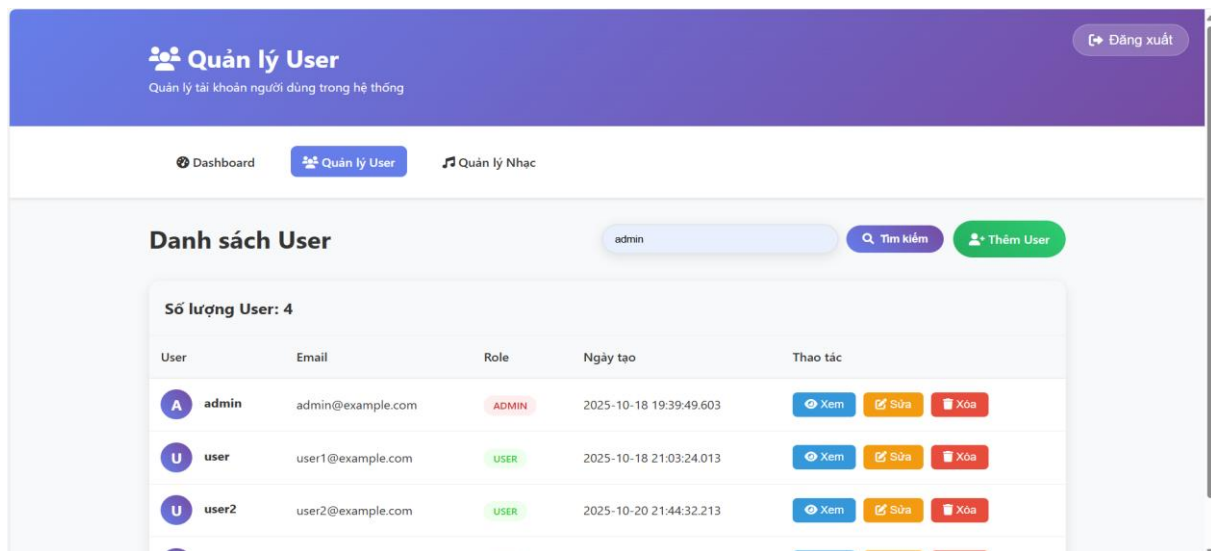


Hình 3. 4 Giao diện User

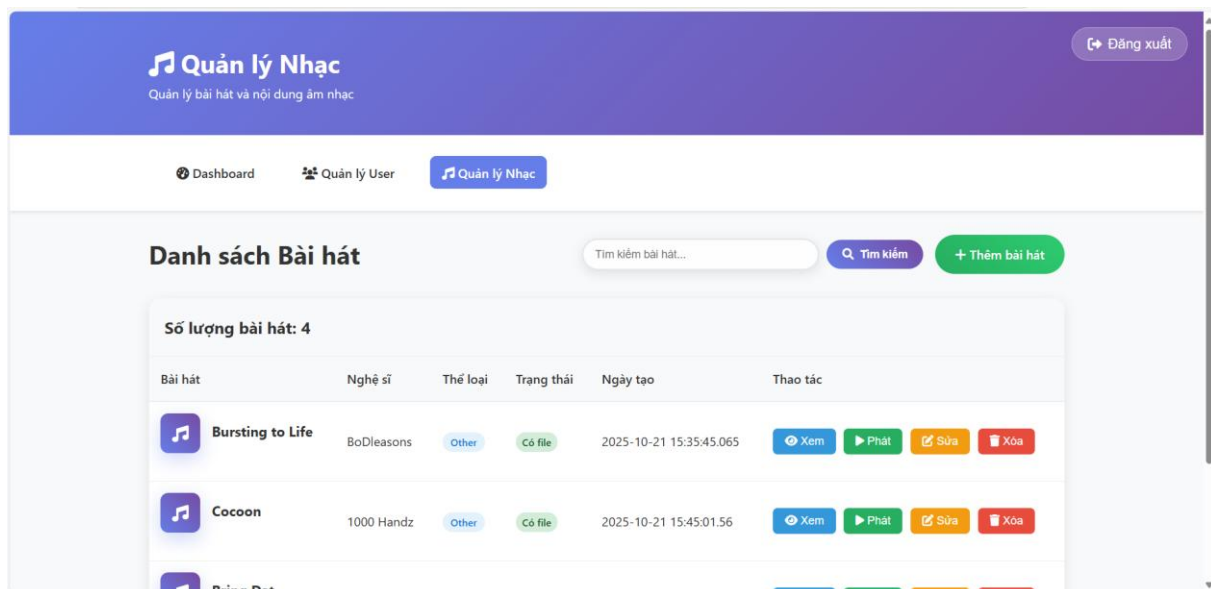
Đối với quản trị viên, hệ thống cung cấp trang Dashboard hiển thị tổng quan số lượng người dùng và bài hát hiện có. Chức năng quản lý người dùng bao gồm xem danh sách, tìm kiếm và xóa người dùng thông qua API /api/admin/users. Bên cạnh đó, quản trị viên có thể quản lý danh sách bài hát, theo dõi trạng thái xử lý HLS, phát thử và xóa bài hát. Các tác vụ quản trị đều được bảo vệ thông qua cơ chế điều hướng an toàn bằng token, đồng thời hệ thống hỗ trợ chức năng đăng xuất.



Hình 3. 5 Giao diện Admin



Hình 3. 6 Giao diện quản lý User (Admin)



Hình 3. 7 Giao diện quản lý Nhạc (Admin)

Hệ thống sử dụng các API chính như `/api/auth/login`, `/api/auth/register`, `/api/auth/me` cho xác thực người dùng, cùng với các API `/api/music`, `/api/music/{id}`, `/api/music/search`, `/api/music/{id}/upload` cho việc quản lý bài hát. Phía quản trị viên có thêm các API `/api/admin/users`, `/api/admin/users/{id}` (DELETE) và `/api/admin/music`, `/api/admin/music/{id}` (DELETE) để phục vụ công tác quản trị.

Về phần phát nhạc trực tuyến, ứng dụng tự động tạo các tệp master.m3u8 cùng các phiên bản nhạc với bitrate 128k và 256k nhờ công cụ ffmpeg. Các tệp này được phục vụ thông qua cấu hình `StaticResourceConfig`, giúp đảm bảo việc phát nhạc ổn định và liên tục trên trình phát.

Name	Date modified	Type	Size
128k	10/22/2025 2:13 AM	File folder	
256k	10/22/2025 2:13 AM	File folder	
master.m3u8	10/22/2025 2:13 AM	M3U8 File	1 KB

Hình 3. 8 Thư mục lưu trữ các tệp âm thanh đã được xử lý

3.3. Đánh giá hệ thống

Hệ thống đã đạt được các mục tiêu đề ra trong giai đoạn thiết kế ban đầu, bao gồm phát nhạc trực tuyến ổn định, quản lý tài khoản người dùng và phân quyền truy cập. Các chức năng hoạt động đúng yêu cầu, giao diện thân thiện và dễ sử dụng.

Về bảo mật, hệ thống áp dụng cơ chế xác thực JWT kết hợp với phân quyền theo vai trò người dùng và quản trị viên nhằm hạn chế truy cập trái phép. Tuy nhiên, token hiện đang được truyền qua URL trong một số chức năng quản trị, đây chỉ là giải pháp tạm thời và cần được cải thiện bằng cách chuyển sang sử dụng cookie HttpOnly hoặc Authorization Header để tăng mức độ an toàn.

Về hiệu năng, hệ thống vận hành ổn định, tốc độ phát nhạc mượt mà nhờ áp dụng công nghệ HLS. Trong tương lai, có thể cải thiện hiệu suất hơn nữa bằng cách sử dụng cache hoặc CDN để giảm tải cho máy chủ.

Tuy nhiên, hệ thống vẫn còn một số hạn chế như chưa tích hợp thuật toán gợi ý nhạc dựa trên hành vi người dùng, chưa hỗ trợ phát nhạc offline, và cơ chế xác thực bằng token URL cần được thay đổi để đảm bảo tính bảo mật cao hơn.

CHƯƠNG 4 KẾT LUẬN VÀ KIẾN NGHỊ

4.1. Kết luận

Đề tài “Ứng dụng phát trực tuyến nhạc với đề xuất tự động dựa trên sở thích người dùng” đã hoàn thành đúng theo mục tiêu đặt ra. Hệ thống được xây dựng bằng Java Spring Boot, kết hợp với SQL Server và giao thức HLS, cho phép phát nhạc trực tuyến mượt mà, ổn định và có khả năng mở rộng.

Ứng dụng đã triển khai thành công các chức năng chính như: đăng ký/đăng nhập, xác thực bằng JWT, phát nhạc qua HLS.js, quản lý danh sách phát, tìm kiếm và trang quản trị dành cho admin. Giao diện được thiết kế bằng Thymeleaf, thân thiện và dễ sử dụng.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, tốc độ phát nhạc nhanh, đảm bảo yêu cầu về hiệu năng và bảo mật cơ bản. Việc áp dụng công nghệ HLS cùng cấu trúc cơ sở dữ liệu hợp lý giúp hệ thống vận hành trơn tru và có tiềm năng phát triển mạnh hơn trong tương lai.

Đồ án cũng giúp nhóm củng cố kiến thức về lập trình mạng, cơ sở dữ liệu, bảo mật web và thiết kế hệ thống phân tầng, đồng thời rèn luyện kỹ năng làm việc nhóm và xử lý tình huống thực tế trong quá trình phát triển phần mềm.

4.2. Kiến nghị

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu cơ bản về chức năng phát nhạc trực tuyến, nhóm nhận thấy vẫn còn nhiều hướng có thể tiếp tục nghiên cứu và hoàn thiện trong tương lai. Trước hết, nhóm kiến nghị cần bổ sung và triển khai tính năng gợi ý nhạc tự động, đây là mục tiêu ban đầu của đề tài nhưng chưa được thực hiện trong phiên bản hiện tại. Tính năng này có thể được xây dựng dựa trên các thuật toán học máy như Collaborative Filtering hoặc Content-Based Filtering, nhằm phân tích lịch sử nghe nhạc và sở thích của người dùng để đưa ra những đề xuất phù hợp hơn.

Bên cạnh đó, nhóm đề xuất tăng cường cơ chế bảo mật của hệ thống, đặc biệt là trong quá trình xác thực người dùng. Việc chuyển từ truyền token qua URL sang sử dụng cookie bảo mật (HttpOnly) sẽ giúp nâng cao mức độ an toàn và hạn chế nguy cơ bị tấn công hoặc đánh cắp thông tin.

Ngoài ra, nhóm cũng khuyến nghị phát triển phiên bản ứng dụng di động trên các nền tảng Android và iOS, nhằm đáp ứng nhu cầu nghe nhạc mọi lúc, mọi nơi của người dùng, đồng thời mở rộng khả năng tiếp cận của hệ thống. Bên cạnh đó, việc tối ưu hiệu năng thông qua tích hợp CDN hoặc bộ nhớ đệm cũng là hướng quan trọng để giảm tải cho máy chủ và cải thiện tốc độ phát nhạc trong môi trường có nhiều người dùng truy cập đồng thời.

Cuối cùng, để tăng tính tương tác và trải nghiệm người dùng, nhóm đề xuất bổ sung các tính năng mở rộng như bình luận, đánh giá bài hát, chia sẻ danh sách phát hoặc gợi ý nhạc theo tâm trạng, thời gian trong ngày. Nếu được tiếp tục nghiên cứu và đầu tư phát triển, hệ thống có thể trở thành một nền tảng phát nhạc trực tuyến hoàn thiện, thân thiện và cá nhân hóa hơn, góp phần nâng cao chất lượng ứng dụng công nghệ trong lĩnh vực giải trí số tại Việt Nam.

TÀI LIỆU THAM KHẢO

- [1] Apple Inc. (2023). *HTTP Live Streaming Overview*. Apple Developer Documentation.
Truy cập tại: https://developer.apple.com/documentation/http_live_streaming
- [2] Baeldung. (2023). *Guide to JPA and Hibernate Relationships*.
Truy cập tại: <https://www.baeldung.com/hibernate-one-to-many>
- [3] Freeman, A. (2021). *Pro ASP.NET Core 6*. Apress, New York.
- [4] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston.
- [5] HLS.js Project. (2023). *HLS.js Documentation*.
Truy cập tại: <https://hlsjs.dev>
- [6] Microsoft. (2023). *SQL Server Documentation*. Microsoft Learn.
Truy cập tại: <https://learn.microsoft.com/en-us/sql/sql-server>
- [7] Microsoft. (2023). *Spring Boot Documentation*. Microsoft Learn.
Truy cập tại: <https://learn.microsoft.com/en-us/spring-boot>
- [8] Microsoft. (2023). *Spring Security Documentation*. Microsoft Learn.
Truy cập tại: <https://learn.microsoft.com/en-us/spring-security>
- [9] Microsoft. (2023). *Thymeleaf Integration with Spring Boot*. Microsoft Learn.
Truy cập tại: <https://learn.microsoft.com/en-us/spring-boot/web>
- [10] Mux. (2023). *FFmpeg for Adaptive Bitrate Streaming*.
Truy cập tại: <https://www.mux.com/blog/ffmpeg-adaptive-bitrate-streaming>
- [11] OWASP Foundation. (2023). *OWASP Top Ten Web Application Security Risks*.
Truy cập tại: <https://owasp.org/www-project-top-ten>
- [12] ScienceDirect. (2022). *Hybrid Recommendation Systems for Music Streaming Platforms*.
Truy cập tại: <https://www.sciencedirect.com/>